

Quiz 1

- Due No due date
- Points 10
- Questions 10
- Available Jan 16 at 7:10pm - Jan 18 at 11:59pm
- Time Limit None
- Allowed Attempts 3

Instructions

Intro and Universal Approximators

This quiz covers lectures 1 and 2. Several of the questions invoke concepts from the hidden slides in the slide deck, which were not covered in class. So please go over the slides before answering the questions.

You will have three attempts for the quiz. Questions will be shuffled and you will not be informed of the correct answers until after the deadline. While you may discuss the concepts underlying the questions with others, you must solve all questions on your own - see course policy.

Attempt History

	Attempt	Time	Score
KEPT	Attempt 3	58 minutes	9 out of 10
LATEST	Attempt 3	58 minutes	9 out of 10
	Attempt 2	14 minutes	8.5 out of 10
	Attempt 1	326 minutes	8 out of 10

⚠ Correct answers are hidden.

Score for this attempt: 9 out of 10

Submitted Jan 18 at 4:29pm

This attempt took 58 minutes.



Question 1

1 / 1 pts

According to the university's academic integrity policies and 11785 course policies, which of the following practices are **NOT** allowed in this course? Select all that apply.

- ☒ Discuss quiz solutions with another student before the deadline
- ☐ Discussing concepts from class with another student
- ☐ Helping another student debug their code
- ☒ Posting code in a public post on piazza
- ☒ Creating a piazza post asking for help debugging code without prior attempts to debugging it yourself.
- ☐ Asking a TA for help debugging your code

See <https://www.cmu.edu/policies/student-and-student-life/academic-integrity.html> (<https://www.cmu.edu/policies/student-and-student-life/academic-integrity.html>). Other than these restrictions, we highly encourage you to discuss course concepts with other students! Post (reasonable) questions on Piazza, and answer others. Learning is most fun with friends :)



Question 2

1 / 1 pts

What term does the following sentence describe? "A modification of Hebbian learning addressing its lack of weight reduction and its instability."

Hint: See lec 1, "Hebbian Learning" Slide 64-66

- ☐ Parallel distributed processing
- ☐ Turing's A-type machines
- ☐ Lawrence Kubie's loop networks
- ☒ Sanger's rule

Hebbian learning is described in the lecture as fundamentally unstable because weights only increase and are never reduced. The slide then introduces **Sanger's rule**, also called generalized Hebbian learning, as a modification that adds normalization and competition between weights, addressing this instability.



Question 3

1 / 1 pts

Is the following statement true or false? Hebbian learning allows reduction in weights and learning is bounded.

Slide: lec 1, "Hebbian Learning" slide 66

- ☐ True
- ☒ False

If neuron x repeatedly triggers neuron y, the synaptic knob (Weight) connecting x to y gets larger. Hence the weight only increases and a mechanism for weight reduction is not given. Also, the upper bound to which the weight increases to is not defined in the learning, making it unbounded.



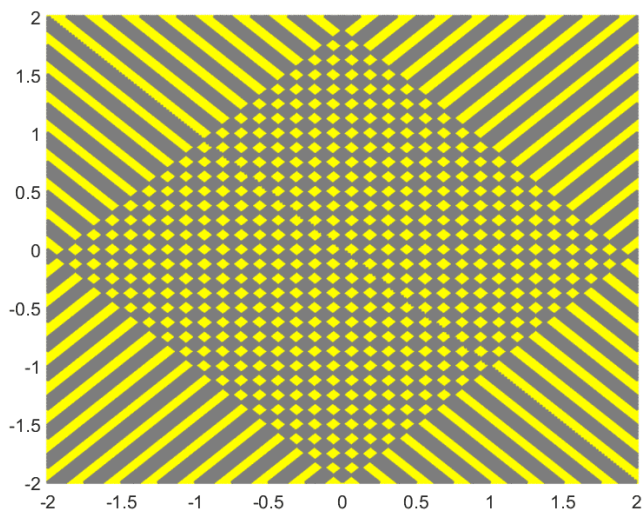
Question 4

1 / 1 pts

We would like to compose an MLP that can represent a three-dimensional version of the cross-hash function, where the input dimension is fixed at 3, shown below (the illustration is 2-dimension because it is difficult to depict the 3-D version of the map). The input is now a three-dimensional vector. The output map comprises alternating cuboids of yellow and grey, where yellow represents a region where the function must output 1, and grey cuboids are regions where it must output 0.

Assume the network is restricted to only two hidden layers (fixed total depth of 3 including output layer). Which of the following statements is true of the number of neurons (not weights) required to model the function below?

(Note: for the definition of network depth, see the lecture 2 recording)



Hint : Slide: lec 2, "Network size?"

- ☐ It is exponential in the number of hyperplanes required to form the cross hash
- ☐ It is linear in the number of hyperplanes required to form the cross hash
- ☒ It is polynomial in the number of hyperplanes required to form the cross hash
- ☐ It is quadratic in the number of hyperplanes required to form the cross hash

It is polynomial in the number of planes, but exponential in the dimensionality of the space.

For N-dimensional inputs, you can have at most N parallel (non-intersecting) hyperplanes in each dimension. If you have N hyperplanes on each of 2 dimensions, they will intersect N^2 times in a crosshash. Each intersection corresponds to one square, so you get up to N^2 squares.

In this question the input dimension is fixed at three. With N hyperplanes per dimension, the number of cuboidal regions scales as N^3 , which is polynomial in N .

Incorrect



Question 5

0 / 1 pts

Is the following statement true or false: for any Boolean function on the real plane which outputs a 1 within a single connected but NON-CONVEX region (*i.e.* not a convex polygon), you need at least three layers (including the output layer) to model the function EXACTLY.

Hint: Lecture 2 slides "Complex decision boundaries"

- ☒ True
- ☐ False

A 5-point star can be exactly modelled with two layers (1 hidden + 1 output). We would get this by using a threshold of 4 at the output neuron. A 5-point star is not convex.



Question 6

1 / 1 pts

Although we haven't yet covered training of neural networks in the lectures, we can give you this advance bit of information: The number of required training inputs to train a network properly is monotonically related to the number of parameters.

In general, as depth increases, and because deeper networks require exponentially fewer parameters to represent the same function, how does the amount of training data required change?

Slide: lec 2, "The challenge of depth" slides 67-68

- ☐ Increases quadratically
- ☐ Decreases quadratically
- ☐ Increases exponentially
- ☒ Decreases exponentially

In general, for a given function, deeper networks will require exponentially fewer parameters than shallower ones to model the function accurately (exactly, or with arbitrary precision). Since the number of required training inputs is monotonically related to the number of parameters required, the number of required training observations therefore decreases exponentially as depth increases.



Question 7

1 / 1 pts

Suppose the data used in a classification task is 10-dimensional and positive in all dimensions. You have two neural networks. The first uses threshold activation functions for hidden layers, and the second uses softplus activation functions for hidden layers. In both networks, there are 2000 neurons in the first hidden layer, 8 neurons in the second hidden layer, and a huge number of neurons for all later layers. It turns out that the first network can never achieve perfect classification. What about the second network?

Hint: A layer with 8 neurons effectively projects the data onto a 8-dimensional surface of the space. The input is 10-dimensional.

Lec 2 slides 138-150

☐ It too is guaranteed to be unable to achieve perfect classification.

☐

Assuming that layers 3 and above are so expressive that they never bottleneck the flow of information, the second network will be able to achieve perfect classification.

☐

It might fail for some data sets, not only because a bottleneck can occur, but also because the 2000 neurons in the first hidden layer could bottleneck the flow of information if the classification task is sufficiently complex.

☒

It might fail for some data sets, since the 8 neurons in the second hidden layer could bottleneck the flow of information. In that case, the sizes of layers 3 and above don't matter.

Answer: B. The 8-neuron layer reduces the dimensionality of the data from 10 to 8. This results in a loss of dimensionality, and potentially, of information.

Regardless of the activations, such a loss of dimensionality can result in loss of key information which prevent perfect classification, unless the data themselves actually live in an 8-dimensional surface in the 10-dimensional space. The fact that the threshold activation network fails to achieve perfect accuracy in spite of having a large number of initial feature detectors suggests that the data do not in fact live on an 8-dimensional surface, although we cannot be certain of it.

⋮

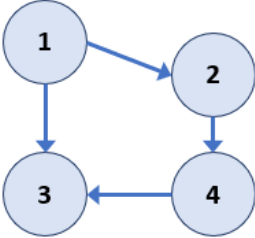
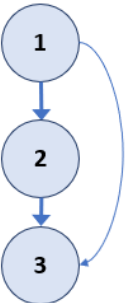
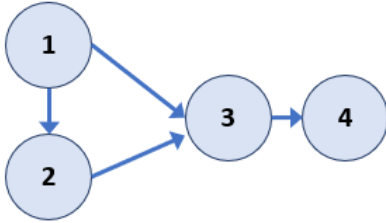
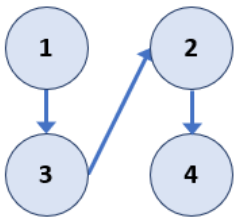
Question 8

1 / 1 pts

Which of the following NNs have a depth of 3? (Choose all that apply)

(Note: for the definition of network depth, see the lecture 2 recording)

Hint: lec 2, "Deep Structures", Slides: 17-18



Depth is defined as longest path from source to sink.

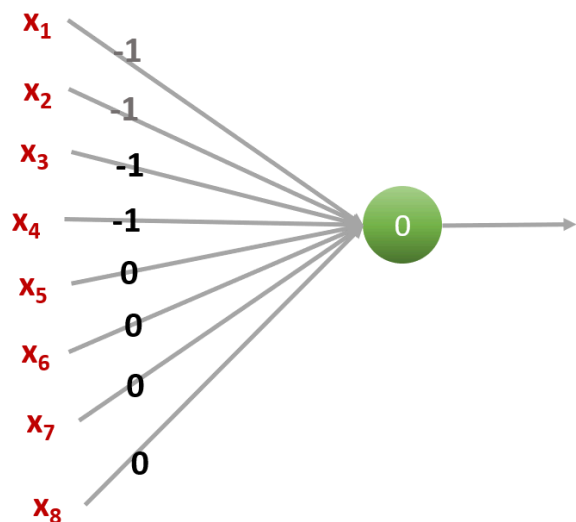
- a) longest path: 1->2->3 (Depth 2)
- b) longest path: 1->2->4->3 (Depth 3)
- c) longest path 1->3->2->4 (Depth 3)
- d) longest path: 1->2->3->4 (Depth 3)



Question 9

1 / 1 pts

Under which conditions will the perceptron graph below fire? Note that ~ is NOT. (select all that apply)



Slide: lec 2, "Perceptron as a Boolean gate", slides 26-30

- ☐ $x_1 \& x_2 \& x_3 \& x_4$
- ☒ fires only if x_1, x_2, x_3, x_4 are all 0, regardless of $x_5 \dots x_8$
- ☒ $\sim x_1 \& \sim x_2 \& \sim x_3 \& \sim x_4$
- ☐ Never fires

The number above each connection is the connection's weight w_i , which is multiplied to its corresponding X_i input (either 0 or 1). For the perceptron to fire, the sum of all $w_i * X_i$ must be $\geq$ to the number in the circle.

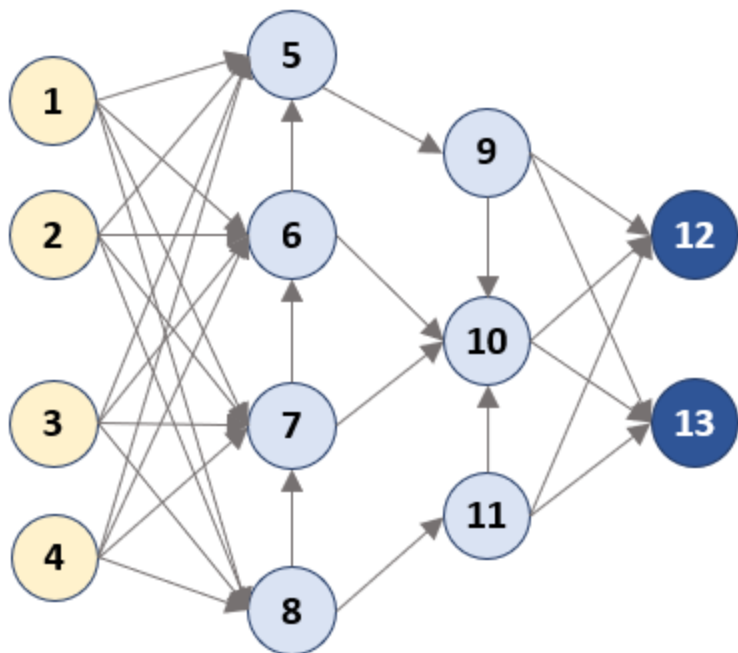
As $w_5 \sim w_8$ are 0, any activations here won't influence firing. But if at least one of $[X_1, X_2, X_3, X_4]$ are 1, the perceptron will not fire, as the total will never be $\geq$ the threshold 0.

$(\sim x_1 \& \sim x_2 \dots)$ is only true when $x_1 \dots x_4$ are all zero.



Question 10

1 / 1 pts



If the yellow nodes are inputs (not neurons) and the dark blue nodes are outputs, which neurons are in layer 2?

(Note: for the definition of network depth and layer number, see the lecture 2 recording)

Hint: lec 2, slides 19-20 on "What is a layer"

- ☒ 7, 11
- ☐ 9, 10, 11
- ☐ 8
- ☐ 5,6,7,8

Pay attention to the definitions. Definition of layer #: the neurons reachable (with the same number of edges) along the longest path from source to sink. If a node is reachable multiple times along the longest path, its layer # is the max of those reachable cases (i.e. if some node D was reachable along the longest path 3 times [1,2,3] along the longest path, it's layer number is 3).

The longest path would be from any input -> 8 -> 7 -> 6 -> 5 -> 9 -> 10 -> any output. Node 8 would count as layer 1, as you traverse 1 edge along the longest path to get there. While it is possible to visit 5,6,7,8 via one edge, they are visited later along the longest path. Layer 2 would be 7 AND 11, as 11 is not visited later along the longest path, but is still reachable with the same number of edges.

Quiz Score: 9 out of 10